🧠 MASTER EXECUTION BRIEF

From Research-Grade Quant System → Production SaaS

Core principles (non-negotiable):

- No data leakage
- Deterministic behavior
- Idempotent daily runs
- Reproducible artifacts
- Defensive capital preservation
- Observable system (logs, metrics, audits)

🔒 PHASE A — Production Bedrock (DO THIS FIRST)

> *"If this breaks, everything breaks."*

🎯 Goal

Turn your inference system into a **stateful, deterministic trading engine** that can run unattended every trading day.

A1. Runtime Configuration & Environment Hardening

Deliverables

- `.env` driven config
- Strict environment separation

Actions

- Create `config/` module:

    - `base.py`
    - `dev.py`
    - `prod.py`
- Centralize:
    - universe
    - transaction costs
    - max leverage
    - capital limits
- Explicit timezone handling (`US/Eastern`)
- Freeze Python + library versions (`poetry.lock` or `requirements.txt`)

Tests

- Config loads deterministically
- Dev/prod parity checks

A2. State Persistence Layer (CRITICAL)

This is where most ML trading projects fail.

State you must persist:

- Open positions (symbol, size, entry price)
- Cash balance
- Last processed date
- Realized PnL
- Unrealized PnL

- Previous signals (audit trail)

Design

```
state/
  ├── portfolio_state.json
  ├── positions.csv
  ├── trades.csv
  ├── daily_snapshots/
```

Rules

- Never overwrite → append + version
- Atomic writes only
- Read-modify-write cycle per trading day

Tests

- Restart simulation mid-week → identical result
- Re-run same date → no duplicate trades

A3. Capital Allocation Logic (Minimal but Correct)

Do NOT over-engineer yet.

Initial rules (production-safe):

- Max % capital per position (e.g. 10%)
- Max number of concurrent positions
- Confidence-weighted sizing
- Risk gating:
    - No trades on extreme volatility days
    - Flat if ensemble says HOLD

Deliverable
`capital_allocator.py`

Test scenarios

- 0 signals → no trades
- 10 BUY signals → capped allocation
- Confidence = 0.51 vs 0.95 → different sizing

A4. Execution Simulator (Not Broker Yet)

Purpose: ensure financial realism

Actions

- Explicit order objects:
  - symbol
  - side
  - quantity
  - expected price
- Apply:
  - slippage
  - transaction costs
- Update portfolio state

Tests

- Round-trip trade cost sanity check
- No negative cash allowed
- Position reconciliation

⏱️ PHASE B — Automation & Scheduling

> *"If humans must press run, it's not production."*

B1. Trading Day Scheduler (Cron)

Design

- Script: `run_daily.py`
- Guard rails:
  - Check market open
  - Check already run today
  - Fail safely

Cron Example

```
0 18 * * 1-5 python run_daily.py >> logs/daily.log
```

Actions

- Add holiday calendar check
- Add retry logic
- Exit codes for monitoring

Tests

- Run twice same day → no duplicate trades
- Run on weekend → no-op

B2. Daily Artifacts (Immutable)

Every run produces:

```
artifacts/YYYY-MM-DD/
  ├── signals.json
  ├── explanations/
  ├── portfolio_snapshot.json
  ├── trades.csv
  ├── metrics.json
```

This is your compliance layer.

📊 PHASE C — Monitoring, Logging & Auditing

> *"If you can't explain what happened, you can't scale."*

C1. Structured Logging

Implement

- `loguru` or `structlog`
- JSON logs only
- Log levels:
  - INFO: signals, trades
  - WARNING: skipped trades
  - ERROR: failures

Log fields

- run_id
- date
- symbol
- signal
- confidence

- capital_used

C2. Outcome Tracking

Track per-signal:

- Signal date
- Entry price
- Exit price
- Holding period
- Realized return

Deliverable

- `signal_outcomes.csv`

This enables:

- Live accuracy tracking
- Confidence calibration
- Marketing proof later

🌐 PHASE D — FastAPI Hardening (Product Core)

> *"APIs are contracts, not scripts."*

D1. API Structure

Endpoints

- `GET /health`

- `GET /latest_signal`
- `GET /portfolio`
- `GET /history?symbol=`
- `POST /predict` (internal use)

Rules

- Read-only for users
- No model mutation via API
- Cached responses where possible

D2. Security Basics

- API keys (header based)
- Rate limiting
- Request logging
- Timeout protection

D3. API Tests

- Pytest + FastAPI TestClient
- Snapshot tests for JSON schema
- Load test (locust or k6)

🔐 PHASE E — Authentication & Access Control

> *This is what turns a project into a business.*

E1. Auth

- Supabase or Auth0
- JWT verification middleware

E2. Access Tiers

- Free:
    - delayed signals
    - limited universe
- Pro:
    - live signals
    - full explanations

🖥️ PHASE F — Frontend Dashboard

> *This is where users fall in love.*

Pages

- Signals (today)
- Portfolio
- Performance
- Explainability
- Account

Principles

- No clutter
- Finance-grade visuals
- Trust first, hype never

💳 PHASE G — Monetization

- Stripe subscriptions
- Webhooks
- Tier enforcement at API layer

📦 PHASE H — Deployment

Stack

- Backend: Render / Fly.io
- Frontend: Vercel
- Storage: S3 compatible
- Secrets: environment only

CI/CD

- Lint
- Tests
- Build
- Deploy

🧪 PHASE I — Heavy Testing (MANDATORY)

This is what separates professionals.

Test Types

- Unit (features, models)
- Integration (daily run)
- Regression (equity curve lock)
- Chaos tests (missing data)
- Restart tests (state recovery)

🧠 PHASE J — Proof, Branding & Career Leverage

- Demo video
- Technical blog series
- Public metrics dashboard
- Polished README
- Architecture diagrams

🏁 FINAL RESULT

You will have:

- A **real trading engine**
- A **deployable SaaS**
- A **research-credible system**
- A **career-defining project**